

## RAPPORT TECHNIQUE

Module : Bases de données réparties

---

# Conception et mise en œuvre d'un système de bases de données réparties

Fragmentation horizontale, synchronisation  
et optimisation sous Oracle 23

---

**Application : plateforme de commerce « EShop »**

**Auteur** Ilyass Bougati, Abdellah Darni

**Technologies** Oracle Database 23 Free, Docker Compose,  
PL/SQL, Next.js, node-oracledb

**Date** 8 juin 2026

## Table des matières

|           |                                                                    |           |
|-----------|--------------------------------------------------------------------|-----------|
| <b>1</b>  | <b>Introduction et objectifs</b>                                   | <b>2</b>  |
| 1.1       | Contexte fonctionnel . . . . .                                     | 2         |
| 1.2       | Objectifs techniques . . . . .                                     | 2         |
| 1.3       | Pile technologique . . . . .                                       | 2         |
| <b>2</b>  | <b>Architecture générale</b>                                       | <b>2</b>  |
| 2.1       | Vue d'ensemble . . . . .                                           | 2         |
| 2.2       | Les nœuds du système . . . . .                                     | 3         |
| 2.3       | Création des bases enfichables (PDB) . . . . .                     | 3         |
| 2.4       | Orchestration par Docker Compose . . . . .                         | 4         |
| 2.5       | Résolution de noms et liens de base de données . . . . .           | 4         |
| 2.6       | Utilisateurs, habilitations et communication inter-nœuds . . . . . | 5         |
| 2.7       | Les deux scénarios de fragmentation . . . . .                      | 6         |
| <b>3</b>  | <b>Modèle de données</b>                                           | <b>7</b>  |
| <b>4</b>  | <b>Fragmentation horizontale</b>                                   | <b>8</b>  |
| 4.1       | Principe . . . . .                                                 | 8         |
| 4.2       | Scénario 1 : fragmentation par catégorie . . . . .                 | 9         |
| 4.3       | Scénario 2 : fragmentation par volume . . . . .                    | 9         |
| 4.4       | Construction des fragments et intégrité référentielle . . . . .    | 9         |
| <b>5</b>  | <b>Procédures stockées de chaque site (Q2 et Q3)</b>               | <b>9</b>  |
| 5.1       | Insertion avec rapatriement des parents . . . . .                  | 10        |
| 5.2       | Suppression en cascade « ascendante » . . . . .                    | 10        |
| 5.3       | Habilitations . . . . .                                            | 11        |
| <b>6</b>  | <b>Synchronisation par déclencheurs (Q4)</b>                       | <b>11</b> |
| 6.1       | Routage à l'insertion . . . . .                                    | 11        |
| 6.2       | Deux décisions de conception déterminantes . . . . .               | 11        |
| 6.3       | Le cas subtil de la mise à jour . . . . .                          | 12        |
| <b>7</b>  | <b>Requêtes distribuées et optimisation (Q5 et Q6)</b>             | <b>13</b> |
| 7.1       | Analyse et optimisation d'une requête locale (Q5) . . . . .        | 13        |
| 7.2       | Reconstruction d'une vue globale (Q6) . . . . .                    | 13        |
| <b>8</b>  | <b>Nœud de sauvegarde</b>                                          | <b>14</b> |
| <b>9</b>  | <b>Tableau de bord web</b>                                         | <b>14</b> |
| <b>10</b> | <b>Démonstration et captures d'écran</b>                           | <b>15</b> |
| 10.1      | Conteneurs Docker en cours d'exécution . . . . .                   | 16        |
| 10.2      | Tableau de bord web . . . . .                                      | 16        |
| 10.3      | Tests de manipulation des données . . . . .                        | 16        |
| <b>11</b> | <b>Avantages de l'architecture</b>                                 | <b>16</b> |
| <b>12</b> | <b>Inconvénients et limites</b>                                    | <b>18</b> |
| <b>13</b> | <b>Pistes d'amélioration</b>                                       | <b>19</b> |
| <b>14</b> | <b>Conclusion</b>                                                  | <b>19</b> |

# 1 Introduction et objectifs

Ce rapport présente la conception et la réalisation d'un **système de gestion de bases de données réparties** appliqué à une plateforme de commerce électronique fictive nommée *EShop*. Le projet illustre les concepts fondamentaux des bases de données distribuées : la *fragmentation horizontale* des données sur plusieurs sites, la *transparence de localisation* pour l'application cliente, la *synchronisation* automatique entre un nœud central et des nœuds satellites, et enfin l'*optimisation* des requêtes locales et distribuées.

## 1.1 Contexte fonctionnel

Le schéma applicatif modélise une boutique en ligne classique : des clients passent des commandes, chaque commande étant détaillée en lignes de commande qui référencent des produits, eux-mêmes classés par catégorie et rattachés à des fournisseurs. La table volumineuse et au cœur de l'activité est **LIGNECOMMANDES** : c'est précisément cette table que l'on cherche à *fragmenter* et à *répartir* sur plusieurs serveurs.

## 1.2 Objectifs techniques

Le cahier des charges se décline en six questions (Q1 à Q6), toutes couvertes par l'implémentation :

- Q1** Définir le schéma global et fragmenter horizontalement la table **LIGNECOMMANDES** selon des prédicats métier.
- Q2** Doter chaque site de procédures stockées d'insertion, de suppression et de mise à jour sur son fragment local.
- Q3** Garantir l'intégrité référentielle de chaque fragment (clés primaires, clés étrangères, rapatriement des lignes parentes).
- Q4** Synchroniser automatiquement les sites depuis le nœud global au moyen de déclencheurs (*triggers*).
- Q5** Analyser et optimiser une requête analytique au moyen de plans d'exécution (**EXPLAIN PLAN**) et d'index.
- Q6** Reconstruire une vue globale (chiffre d'affaires par catégorie) à partir des fragments distants via des liens de base de données.

## 1.3 Pile technologique

L'ensemble repose sur **Oracle Database 23 Free** (image `gvenzl/oracle-free`), chaque nœud étant un conteneur **Docker** orchestré par **Docker Compose**. La logique métier est écrite en **PL/SQL**. Un **tableau de bord web** (Next.js + `node-oracledb`) complète le dispositif pour visualiser en temps réel l'état des nœuds et le routage des données.

# 2 Architecture générale

## 2.1 Vue d'ensemble

L'architecture suit un modèle **hub-and-spoke** (étoile) : un nœud *global* central détient le jeu de données complet et joue le rôle d'unique point d'écriture ; autour de lui gravitent des nœuds *site* qui n'hébergent qu'un fragment des données, ainsi qu'un nœud de *sauvegarde*. Tous les conteneurs communiquent sur un réseau Docker privé, **maroc-db-net**, et se résolvent mutuellement par des liens de base de données Oracle (*database links*).

## 2.2 Les nœuds du système

Chaque nœud est une base de données enfichable (*Pluggable Database*, PDB) hébergée dans son propre conteneur. Le tableau 1 récapitule la topologie pour le scénario 1.

| Nœud       | Conteneur     | Port hôte | Rôle                                            |
|------------|---------------|-----------|-------------------------------------------------|
| Global     | oracle-global | 1521      | Jeu de données complet, déclencheurs de synchro |
| Site 1     | oracle-site-1 | 1522      | Fragment R1                                     |
| Site 2     | oracle-site-2 | 1523      | Fragment R2                                     |
| Sauvegarde | oracle-backup | 1524      | Copie complète, rechargement horaire            |

TABLE 1 – Topologie des nœuds (scénario 1). Le scénario 2 est déployé sur une pile parallèle aux ports 1531–1533.

## 2.3 Création des bases enfichables (PDB)

L'image `gvenzl/oracle-free:23` fournit une base *multilocataire* (*multitenant*) : un conteneur racine `CDB$ROOT` pouvant héberger plusieurs bases enfichables (*Pluggable Databases*). Plutôt que d'utiliser la PDB livrée par défaut, **chaque nœud crée la sienne** au premier démarrage (`G_PDB`, `S1_PDB`, `S2_PDB`, `B_PDB`). Cela donne à chaque base un nom de service explicite (ce qui rend les alias TNS et les liens lisibles) et isole complètement les jeux de données les uns des autres.

La procédure est identique sur les quatre nœuds (seul le nom change). On se place dans le conteneur racine, on *clone* la PDB modèle `pdbseed` en recopiant ses fichiers via `FILE_NAME_CONVERT`, on déclare un administrateur local `pdb_admin`, puis on ouvre la PDB et on **sauvegarde son état** afin qu'elle se rouvre automatiquement à chaque redémarrage du conteneur.

```

1  -- 1) Se placer dans le conteneur racine de la base multilocataire
2  ALTER SESSION SET CONTAINER = CDB$ROOT;
3
4  -- 2) Cloner la PDB modele (pdbseed) en une nouvelle base enfichable
5  CREATE PLUGGABLE DATABASE G_PDB
6    ADMIN USER pdb_admin IDENTIFIED BY Admin123
7    ROLES = (DBA)
8    DEFAULT TABLESPACE USERS
9    DATAFILE '/opt/oracle/oradata/FREE/G_PDB/users01.dbf'
10   SIZE 250M AUTOEXTEND ON
11   FILE_NAME_CONVERT = (
12     '/opt/oracle/oradata/FREE/pdbseed/',    -- source : la PDB modele
13     '/opt/oracle/oradata/FREE/G_PDB/'      -- cible  : les fichiers de G_PDB
14   );
15
16  -- 3) Ouvrir la PDB et l'enregistrer aupres de l'auditeur (listener)
17  ALTER PLUGGABLE DATABASE G_PDB OPEN;
18  ALTER SYSTEM REGISTER;
19
20  -- 4) Conserver l'etat OPEN au redemarrage du conteneur
21  ALTER PLUGGABLE DATABASE G_PDB SAVE STATE;
22
23  -- 5) Basculer dans la nouvelle PDB pour y creer le schema
24  ALTER SESSION SET CONTAINER = G_PDB;

```

Extrait 1 – Création d'une base enfichable depuis `CDB$ROOT` (extrait de `01_create_schema.sql`, nœud global).

Deux instructions méritent attention. `ALTER SYSTEM REGISTER` force l'enregistrement immédiat du service auprès de l'*auditeur*, sans quoi un lien de base de données entrant pourrait ne pas résoudre le service pendant la première minute. `SAVE STATE` évite d'avoir à rouvrir manuellement la PDB (elle est `MOUNTED` mais fermée par défaut après un redémarrage).

## 2.4 Orchestration par Docker Compose

Le fichier `compose.yaml` décrit les conteneurs, leurs volumes persistants, leurs limites de ressources, et surtout l'**ordre de démarrage**. Les scripts SQL placés dans `scripts-*/` sont montés dans `/container-entrpoint-initdb.d/` et exécutés automatiquement au premier démarrage de chaque conteneur.

Le point délicat est la **dépendance de démarrage** : les sites ne doivent s'initialiser qu'une fois le nœud global prêt, car leurs scripts rapatrient des données depuis celui-ci. Cela est garanti par la combinaison d'un `healthcheck` sur le global et d'une clause `depends_on: condition: service_healthy` sur les sites. Le `healthcheck` interroge une table sentinelle `INIT_COMPLETE`, créée tout à la fin du dernier script du global : tant que la sentinelle est absente, le global est considéré comme « non sain » et les sites attendent.

```

1 db-global:
2   image: gvenzl/oracle-free:23-slim-faststart
3   container_name: oracle-global
4   ports:
5     - "1521:1521"
6   volumes:
7     - global-data:/opt/oracle/oradata
8     - ./scripts-global:/container-entrpoint-initdb.d/
9     - ./tnsnames.ora:/opt/oracle/network/admin/tnsnames.ora
10  healthcheck:
11    test:
12      [ "CMD-SHELL",
13        "echo 'SELECT sentinel FROM pdb_admin.INIT_COMPLETE WHERE ROWNUM=1;' \
14          | sqlplus -s s1_user/mon_mdp@//localhost:1521/G_PDB \
15          | grep -q READY || exit 1" ]
16    interval: 30s
17    timeout: 10s
18    retries: 20
19    start_period: 120s
20  networks:
21    - maroc-db-net
22
23 db-site-1:
24   # ...
25   depends_on:
26     db-global:
27       condition: service_healthy  # le site attend que le global soit prêt

```

Extrait 2 – Service du nœud global : `healthcheck` par sentinelle (extrait de `compose.yaml`).

## 2.5 Résolution de noms et liens de base de données

Un fichier `tnsnames.ora` unique est partagé entre tous les conteneurs. Il définit des alias réseau (`GLOBAL_DB_ALIAS`, `S1_ALIAS`, `S2_ALIAS`, `BACKUP_ALIAS`) qui pointent vers les noms d'hôtes Docker. Ces alias sont ensuite utilisés pour créer les *database links*, qui sont le mécanisme par lequel un nœud Oracle accède aux objets d'un autre nœud, comme s'ils étaient locaux.

```

1 -- tnsnames.ora : alias resolu via le DNS interne de Docker
2 GLOBAL_DB_ALIAS =
3   (DESCRIPTION =
4     (ADDRESS = (PROTOCOL = TCP)(HOST = db-global)(PORT = 1521))
5     (CONNECT_DATA = (SERVER = DEDICATED)(SERVICE_NAME = G_PDB)))
6
7 -- Depuis un site, on cree un lien vers le global ...
8 CREATE DATABASE LINK link_to_global
9   CONNECT TO s1_user IDENTIFIED BY mon_mdp
10  USING 'GLOBAL_DB_ALIAS';

```

```

11
12 -- ... et l'on masque la localisation derriere des synonymes :
13 -- une requete sur PRODUITS interroge en realite le noeud global.
14 CREATE OR REPLACE SYNONYM PRODUITS FOR pdb_admin.PRODUITS@link_to_global;

```

Extrait 3 – Alias TNS et création d'un lien de base de données.

L'usage de **synonymes** par-dessus les liens réalise la *transparence de localisation* : le code des procédures d'un site écrit `SELECT ... FROM PRODUITS` sans savoir que la table physique réside sur le nœud global.

## 2.6 Utilisateurs, habilitations et communication inter-nœuds

### Les comptes de chaque nœud

Chaque PDB possède un propriétaire de schéma, `pdb_admin`, créé comme `ADMIN USER` à la naissance de la base (section 2.3). C'est le seul compte puissant, et il n'est utilisé qu'au moment de l'initialisation par les scripts montés dans le conteneur ; le système *en fonctionnement* ne s'en sert jamais à travers le réseau. Toute la communication inter-nœuds passe par des **comptes de service dédiés**, au plus faible privilège possible (tableau 2).

| Compte                 | Réside sur     | Utilisé par     | Privilèges accordés                                                           |
|------------------------|----------------|-----------------|-------------------------------------------------------------------------------|
| <code>pdb_admin</code> | toutes les PDB | scripts d'init  | propriétaire du schéma (DBA local)                                            |
| <code>s1_user</code>   | G_PDB          | lien du Site 1  | SELECT (7 tables), CREATE SESSION, EXECUTE <code>insert_ligne_commande</code> |
| <code>s2_user</code>   | G_PDB          | lien du Site 2  | SELECT (7 tables), CREATE SESSION                                             |
| <code>b_user</code>    | G_PDB          | lien du Backup  | SELECT (7 tables), CREATE SESSION                                             |
| <code>g_user</code>    | S1_PDB, S2_PDB | liens du global | EXECUTE (3 procédures), SELECT (tables de fragment), CREATE SESSION           |

TABLE 2 – Comptes fonctionnels et leurs privilèges. Aucun ne dispose des droits `DBA`, `CREATE TABLE` ou d'écriture directe.

### Le principe du moindre privilège

Chaque compte de service ne reçoit que les droits strictement nécessaires à sa mission, et rien de plus. Les comptes qui ne font que *lire* le nœud global (`s1_user`, `s2_user`, `b_user`) n'obtiennent que `SELECT` sur les tables et `CREATE SESSION` pour ouvrir une connexion : ils ne peuvent ni modifier les données, ni créer d'objet, ni voir les autres schémas. À l'inverse, le compte `g_user`, par lequel le global *écrit* sur les sites, ne reçoit pas l'accès direct aux tables : il n'a que le droit d'`EXECUTE` sur les trois procédures de routage, qui encapsulent et contrôlent toutes les écritures.

```

1  -- Sur le noeud GLOBAL : un compte de lien ne recoit que la LECTURE
2  GRANT SELECT ON pdb_admin.CLIENTS          TO s1_user;
3  GRANT SELECT ON pdb_admin.PRODUITS          TO s1_user;
4  GRANT SELECT ON pdb_admin.LIGNECOMMANDES TO s1_user;  -- ... idem pour les 7 tables
5  GRANT CREATE SESSION                        TO s1_user;
6  GRANT EXECUTE ON pdb_admin.insert_ligne_commande TO s1_user;
7
8  -- Sur chaque SITE : g_user n'exécute QUE les procedures de routage
9  GRANT CREATE SESSION                        TO g_user;
10 GRANT EXECUTE ON pdb_admin.insertligne      TO g_user;
11 GRANT EXECUTE ON pdb_admin.deleteligne      TO g_user;
12 GRANT EXECUTE ON pdb_admin.updateligne      TO g_user;
13 GRANT SELECT ON  pdb_admin.LIGNECOMMANDES1 TO g_user;  -- lecture des fragments

```

Extrait 4 – Habilitations au plus juste : aucun `GRANT ALL` (extraits de `04_create_roles.sql` et des scripts de site).

Une factorisation de ces autorisations dans un rôle `site_role` a été préparée (et laissée en commentaire dans `04_create_roles.sql`) ; la version déployée conserve des `GRANT` explicites par compte, plus lisibles à des fins pédagogiques et auditable d’un coup d’œil.

### Qui se connecte à qui

La communication repose entièrement sur des *database links*, chacun ouvrant une session sur la base distante **au nom d’un compte de service précis**. Le sens des flux est dissymétrique : les sites *tirent* les données de référence depuis le global, tandis que le global *pousse* les écritures vers les sites via les déclencheurs (section 6).

| Lien           | Créé sur | Cible  | Connexion en | Usage                           |
|----------------|----------|--------|--------------|---------------------------------|
| link_to_global | Site 1   | G_PDB  | s1_user      | lecture des parents (synonymes) |
| link_to_global | Site 2   | G_PDB  | s2_user      | lecture des parents (synonymes) |
| link_to_global | Backup   | G_PDB  | b_user       | rechargement horaire            |
| link_to_site1  | Global   | S1_PDB | g_user       | propagation des écritures       |
| link_to_site2  | Global   | S2_PDB | g_user       | propagation des écritures       |

TABLE 3 – Les cinq liens de base de données et le compte de service utilisé par chacun.

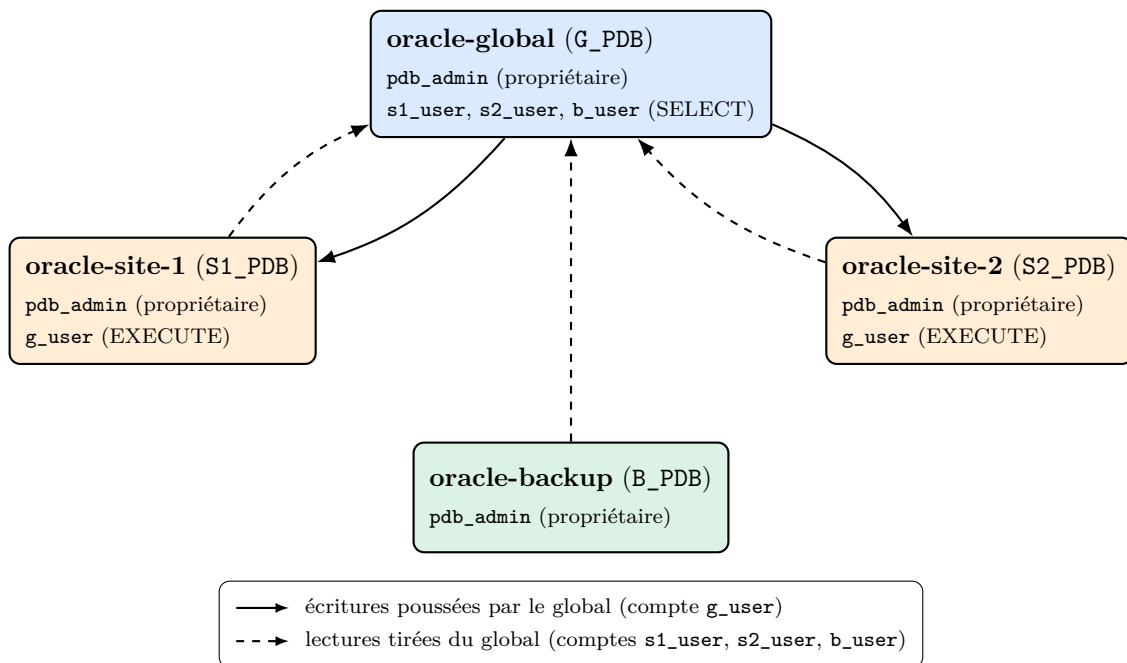


FIGURE 2 – Communication inter-nœuds. Les flèches pleines représentent les écritures poussées par le nœud global vers les sites (via le compte `g_user`) ; les flèches pointillées, les lectures que les autres nœuds tirent du global. Le détail de chaque lien et du compte qu’il emploie figure au tableau 3.

## 2.7 Les deux scénarios de fragmentation

Le projet implémente deux stratégies de fragmentation, déployées sur deux piles indépendantes (répertoires `scenario-1/` et `scenario-2/`). Le tableau 4 les compare ; la section 4 les détaille.

|                      | Scénario 1 (par catégorie)    | Scénario 2 (par volume)         |
|----------------------|-------------------------------|---------------------------------|
| Fragment R1 (Site 1) | IDCATEG=50 $\wedge$ QTÉ>100   | QTÉ $\geq$ 100                  |
| Fragment R2 (Site 2) | IDCATEG=35 $\wedge$ QTÉ>50    | QTÉ < 100                       |
| Disjoints            | oui                           | oui                             |
| Exhaustifs           | <b>non</b> (reste au global)  | <b>oui</b> (partition complète) |
| Donnée de routage    | catégorie (jointure PRODUITS) | QUANTITE (colonne directe)      |

TABLE 4 – Comparaison des deux stratégies de fragmentation horizontale.

### 3 Modèle de données

Le schéma global comprend sept tables. CLIENTS, FOURNISSEURS, EMPLOYES et CATEGORIES sont des tables de référence ; COMMANDES, PRODUITS et LIGNECOMMANDES portent l'activité transactionnelle. Les relations clés sont : une COMMANDE appartient à un CLIENT et à un EMPLOYE ; un PRODUIT relève d'une CATEGORIE et d'un FOURNISSEUR ; une LIGNECOMMANDE relie une COMMANDE à un PRODUIT avec une quantité et une remise. Le rôle de chacune des sept tables est le suivant :

- CLIENTS : fiche des clients (société, contact, adresse, pays...).
- FOURNISSEURS : fiche des fournisseurs approvisionnant les produits.
- EMPLOYES : personnel ayant saisi les commandes.
- CATEGORIES : nomenclature des familles de produits ; sa clé IDCATEG sert de critère de routage au scénario 1.
- COMMANDES : en-têtes de commande (client, employé, dates).
- PRODUITS : catalogue, rattaché à une catégorie et un fournisseur.
- LIGNECOMMANDES : lignes de commande (produit, quantité, remise) ; c'est la **seule table fragmentée** entre les sites.

Les quatre tables de référence portent les clés primaires vers lesquelles pointent les clés étrangères des tables transactionnelles :

```

1 CREATE TABLE CLIENTS (
2     IDCLIENT    NUMBER PRIMARY KEY,
3     CODECLIENT VARCHAR2(100),
4     SOCIETE     VARCHAR2(100) NOT NULL,
5     CONTACT     VARCHAR2(100) NOT NULL,
6     FONCTION    VARCHAR2(100) NOT NULL,
7     ADRESSE     VARCHAR2(100), VILLE VARCHAR2(100), REGION VARCHAR2(100),
8     CP          VARCHAR2(10),  PAYS  VARCHAR2(100),
9     NAISSANCE   DATE,
10    TELEPHONE   VARCHAR2(100), FAX   VARCHAR2(100)
11 );
12
13 CREATE TABLE FOURNISSEURS (
14     IDFOUR      INTEGER PRIMARY KEY,
15     SOCIETEFOUR VARCHAR2(100 CHAR),
16     CONTACTFOUR VARCHAR2(100 CHAR),
17     -- ... autres colonnes : fonction, adresse, ville, region, CP, telephone, fax
18     PAYSFOUR    VARCHAR2(100 CHAR)
19 );
20
21 CREATE TABLE EMPLOYES (
22     IDEMPOYE     INTEGER PRIMARY KEY,
23     NOM          VARCHAR2(100 CHAR),
24     PRENOM       VARCHAR2(100 CHAR),
25     FONCTIONEMPLOYE VARCHAR2(100 CHAR),
26     DATEEMBAUCHE DATE,

```

```

27      -- ... autres colonnes : titre de courtoisie, naissance, adresse, ville, pays, tel
28      VILLEEMPLOYE      VARCHAR2(100 CHAR)
29  );
30
31  CREATE TABLE CATEGORIES (
32      IDCATEG            INTEGER PRIMARY KEY,
33      NOMDECATEGORIE    VARCHAR2(100 CHAR),
34      DESCRIPTION        CLOB
35  );

```

Extrait 5 – Tables de référence du schéma global (extrait de 02\_create\_table.sql).

Les trois tables transactionnelles, qui matérialisent ces relations par des clés étrangères, sont créées ensuite :

```

1  CREATE TABLE COMMANDES (
2      IDCOMMANDE        INTEGER PRIMARY KEY,
3      IDEMPLOYE         INTEGER REFERENCES Employes(IDEMPLOYE),
4      IDCLIENT          INTEGER REFERENCES Clients(IDCLIENT),
5      DATECOMMANDE      DATE,
6      DATELIVRAISON     DATE, CHECK (DATELIVRAISON >= DATECOMMANDE),
7      NMESSAGER         NUMBER(4,0),
8      PORTNUMBER        NUMBER(4,0)
9  );
10
11  CREATE TABLE PRODUITS (
12      IDPRODUIT         INTEGER PRIMARY KEY,
13      DESIGNATION       VARCHAR2(100 CHAR),
14      IDFOUR            INTEGER REFERENCES FOURNISSEURS(IDFOUR),
15      IDCATEG           INTEGER REFERENCES CATEGORIES(IDCATEG),
16      PRIXUNITAIRE      FLOAT,
17      INDISPONIBLE      INTEGER, CHECK (INDISPONIBLE IN (0,1))
18  );
19
20  CREATE TABLE LigneCommandes (
21      IDLIGNECOMMANDE   INTEGER PRIMARY KEY,
22      IDCOMMANDE        INTEGER REFERENCES COMMANDES(IDCOMMANDE),
23      IDPRODUIT         INTEGER REFERENCES PRODUITS(IDPRODUIT),
24      QUANTITE          INTEGER,
25      REMISE            FLOAT
26  );

```

Extrait 6 – Tables transactionnelles du schéma global (extrait de 02\_create\_table.sql).

On notera que la **catégorie d'un produit n'est pas stockée** sur la ligne de commande : elle n'est accessible que par jointure avec PRODUITS. Cette particularité a une conséquence directe sur le scénario 1, où le routage dépend de la catégorie (voir section 6).

## 4 Fragmentation horizontale

### 4.1 Principe

La *fragmentation horizontale* consiste à partitionner une table en sous-ensembles de *lignes* (et non de colonnes) au moyen d'un prédicat de sélection. En algèbre relationnelle, chaque fragment  $R_i$  est défini par une sélection  $\sigma_{P_i}(R)$ . Deux propriétés sont recherchées :

- **Disjonction** :  $\sigma_{P_i}(R) \cap \sigma_{P_j}(R) = \emptyset$  pour  $i \neq j$ , aucune ligne n'est dupliquée entre deux sites.
- **Complétude (ou exhaustivité)** :  $\bigcup_i \sigma_{P_i}(R) = R$ , aucune ligne n'est perdue.

## 4.2 Scénario 1 : fragmentation par catégorie

$$R_1 = \sigma_{IDCATEG=50 \wedge QUANTITE > 100}(\text{LigneCommandes}),$$

$$R_2 = \sigma_{IDCATEG=35 \wedge QUANTITE > 50}(\text{LigneCommandes}).$$

Ces deux fragments sont **disjoints** (les catégories 50 et 35 sont exclusives) mais **non exhaustifs** : toutes les lignes dont la catégorie n'est ni 50 ni 35, ou dont la quantité est trop faible, n'appartiennent à aucun site et ne subsistent que sur le nœud global.

## 4.3 Scénario 2 : fragmentation par volume

$$R_1 = \sigma_{QUANTITE \geq 100}(\text{LigneCommandes}),$$

$$R_2 = \sigma_{QUANTITE < 100}(\text{LigneCommandes}).$$

Cette partition est à la fois disjointe *et* exhaustive : toute ligne appartient à exactement un site. Le routage ne nécessite alors aucune jointure, puisque **QUANTITE** est une colonne directe de la table.

## 4.4 Construction des fragments et intégrité référentielle

Chaque site construit ses tables de fragment par `CREATE TABLE AS SELECT` en filtrant les données rapatriées du global via les synonymes. Comme cette syntaxe **n'hérite pas des contraintes** (clés primaires, clés étrangères), celles-ci sont **reconstruites explicitement**. De plus, pour préserver l'intégrité référentielle, le site n'importe pas seulement les lignes de commande : il importe aussi les **PRODUITS**, **COMMANDES** et **CLIENTS** *associés* (réponse à Q3).

```

1  -- Sous-ensemble des lignes de commande satisfaisant R1
2  CREATE TABLE LIGNECOMMANDES1 AS
3      SELECT lc.*
4      FROM   LIGNECOMMANDES lc
5      JOIN   PRODUITS p ON lc.IDPRODUIT = p.IDPRODUIT
6      WHERE  lc.QUANTITE > 100
7      AND    p.IDCATEG = 50;
8
9  -- La cle primaire n'est PAS heritee du CREATE TABLE AS SELECT
10 ALTER TABLE LIGNECOMMANDES1
11     ADD CONSTRAINT lignecommandes1_pk PRIMARY KEY (IDLIGNECOMMANDE);
12
13 -- On ne rapatrie que les produits/commandes/clients effectivement references
14 CREATE TABLE PRODUITS1 AS
15     SELECT DISTINCT p.*
16     FROM   PRODUITS p
17     JOIN   LIGNECOMMANDES1 lc ON p.IDPRODUIT = lc.IDPRODUIT;
18
19 -- ... puis on retablit les clees etrangeres entre fragments locaux
20 ALTER TABLE LIGNECOMMANDES1
21     ADD CONSTRAINT lignecommandes1_produits1_fk
22     FOREIGN KEY (IDPRODUIT) REFERENCES PRODUITS1(IDPRODUIT);

```

Extrait 7 – Construction du fragment R1 sur le site 1 (extrait de 03\_create\_tables.sql).

## 5 Procédures stockées de chaque site (Q2 et Q3)

Chaque site expose trois procédures PL/SQL (`insertligne`, `deleteligne` et `updateligne`) qui encapsulent toute la logique de manipulation de son fragment. Elles sont appelées à distance par les déclencheurs du nœud global (section 6).

## 5.1 Insertion avec rapatriement des parents

La procédure `insertligne` ne se contente pas d'insérer la ligne : elle **garantit au préalable la présence des lignes parentes**. Si le produit, la commande ou le client correspondant est absent du fragment, il est importé à la volée depuis le global. C'est cette discipline qui maintient l'intégrité référentielle malgré la fragmentation.

```

1 CREATE OR REPLACE PROCEDURE insertligne (
2     p_idlignecommande IN LIGNECOMMANDES1.IDLIGNECOMMANDE%TYPE,
3     p_idcommande      IN LIGNECOMMANDES1.IDCOMMANDE%TYPE,
4     p_idproduit       IN LIGNECOMMANDES1.IDPRODUIT%TYPE,
5     p_quantite        IN LIGNECOMMANDES1.QUANTITE%TYPE,
6     p_remise          IN LIGNECOMMANDES1.REMISE%TYPE
7 ) IS
8     v_count    INTEGER;
9     v_idclient COMMANDES1.IDCLIENT%TYPE;
10 BEGIN
11     -- Le produit existe-t-il déjà localement ? Sinon, on l'importe du global.
12     SELECT COUNT(*) INTO v_count FROM PRODUITS1 WHERE IDPRODUIT = p_idproduit;
13     IF v_count = 0 THEN
14         INSERT INTO PRODUITS1 SELECT * FROM PRODUITS WHERE IDPRODUIT = p_idproduit;
15     END IF;
16
17     -- Idem pour la commande (et le client qu'elle reference)
18     SELECT COUNT(*) INTO v_count FROM COMMANDES1 WHERE IDCOMMANDE = p_idcommande;
19     IF v_count = 0 THEN
20         SELECT IDCLIENT INTO v_idclient FROM COMMANDES WHERE IDCOMMANDE = p_idcommande;
21         SELECT COUNT(*) INTO v_count FROM CLIENTS1 WHERE IDCLIENT = v_idclient;
22         IF v_count = 0 THEN
23             INSERT INTO CLIENTS1 SELECT * FROM CLIENTS WHERE IDCLIENT = v_idclient;
24         END IF;
25         INSERT INTO COMMANDES1 SELECT * FROM COMMANDES WHERE IDCOMMANDE = p_idcommande;
26     END IF;
27
28     INSERT INTO LIGNECOMMANDES1 (IDLIGNECOMMANDE, IDCOMMANDE, IDPRODUIT, QUANTITE, REMISE)
29     VALUES (p_idlignecommande, p_idcommande, p_idproduit, p_quantite, p_remise);
30     COMMIT;
31 EXCEPTION
32     WHEN OTHERS THEN ROLLBACK; RAISE;
33 END insertligne;
34 /

```

Extrait 8 – Insertion avec rapatriement automatique des parents (extrait, site 1).

## 5.2 Suppression en cascade « ascendante »

Symétriquement, `deleteligne` supprime la ligne puis **nettoie les parents devenus orphelins** : si la commande n'a plus aucune ligne, elle est supprimée ; si le client n'a plus aucune commande, il l'est à son tour. Les produits, en revanche, sont des données de référence partagées et ne sont jamais supprimés.

```

1 DELETE FROM LIGNECOMMANDES1 WHERE IDLIGNECOMMANDE = p_idlignecommande;
2
3 -- La commande a-t-elle encore des lignes ?
4 SELECT COUNT(*) INTO v_count FROM LIGNECOMMANDES1 WHERE IDCOMMANDE = v_idcommande;
5 IF v_count = 0 THEN
6     DELETE FROM COMMANDES1 WHERE IDCOMMANDE = v_idcommande;
7     -- Le client a-t-il encore des commandes ?
8     SELECT COUNT(*) INTO v_count FROM COMMANDES1 WHERE IDCLIENT = v_idclient;
9     IF v_count = 0 THEN
10         DELETE FROM CLIENTS1 WHERE IDCLIENT = v_idclient;
11     END IF;

```

```
12 END IF;
```

Extrait 9 – Suppression et nettoyage des parents orphelins (extrait).

### 5.3 Habilitations

Les procédures sont exécutées au travers du lien de base de données par un compte de service dédié, `g_user`. Celui-ci reçoit donc les droits `EXECUTE` sur les trois procédures, `SELECT` sur les tables de fragment, ainsi que `CREATE SESSION`, et rien d'autre. Ce compte est une illustration directe du principe du moindre privilège détaillé en section 2.6.

## 6 Synchronisation par déclencheurs (Q4)

C'est le cœur du système. Trois déclencheurs (*triggers*) posés sur la table `LIGNECOMMANDES` du nœud global interceptent chaque `INSERT`, `DELETE` et `UPDATE` et propagent la modification vers le site propriétaire, en appelant à distance les procédures de la section précédente. L'application n'écrit donc **que sur le global** : le routage est entièrement automatique et transparent.

### 6.1 Routage à l'insertion

Le déclencheur d'insertion résout d'abord la catégorie du produit (absente de la ligne, comme noté plus haut), puis dirige la ligne vers Site 1 et/ou Site 2 selon les prédicats  $R_1$  et  $R_2$ .

```
1 CREATE OR REPLACE TRIGGER SYC_INSERT_LIGNE
2 AFTER INSERT ON pdb_admin.LIGNECOMMANDES
3 FOR EACH ROW
4 DECLARE
5     PRAGMA AUTONOMOUS TRANSACTION;          -- (voir explications ci-dessous)
6     v_idcateg PRODUITS.IDCATEG%TYPE;
7 BEGIN
8     -- La categorie n'est pas sur la ligne : on la lit dans PRODUITS
9     SELECT IDCATEG INTO v_idcateg FROM PRODUITS WHERE IDPRODUIT = :NEW.IDPRODUIT;
10
11     -- Route vers Site 1 si la ligne satisfait R1
12     IF v_idcateg = 50 AND :NEW.QUANTITE > 100 THEN
13         EXECUTE IMMEDIATE 'BEGIN pdb_admin.insertligne@link_to_site1(:1,:2,:3,:4,:5); END;'
14         USING :NEW.IDLIGNECOMMANDE, :NEW.IDCOMMANDE, :NEW.IDPRODUIT,
15             :NEW.QUANTITE, :NEW.REMISE;
16     END IF;
17
18     -- Route vers Site 2 si la ligne satisfait R2
19     IF v_idcateg = 35 AND :NEW.QUANTITE > 50 THEN
20         EXECUTE IMMEDIATE 'BEGIN pdb_admin.insertligne@link_to_site2(:1,:2,:3,:4,:5); END;'
21         USING :NEW.IDLIGNECOMMANDE, :NEW.IDCOMMANDE, :NEW.IDPRODUIT,
22             :NEW.QUANTITE, :NEW.REMISE;
23     END IF;
24 EXCEPTION
25     WHEN NO_DATA_FOUND THEN NULL;          -- produit absent => aucun fragment
26     WHEN OTHERS THEN
27         RAISE_APPLICATION_ERROR(-20010, 'SYC_INSERT_LIGNE: ' || SQLERRM);
28 END SYC_INSERT_LIGNE;
29 /
```

Extrait 10 – Déclencheur de routage à l'insertion, scénario 1 (extrait de `06_create_triggers.sql`).

### 6.2 Deux décisions de conception déterminantes

Le code ci-dessus comporte deux choix techniques qui méritent d'être justifiés, car ils contournent deux erreurs Oracle bien précises.

**1. EXECUTE IMMEDIATE plutôt qu'un appel direct.** Oracle 23c valide les références aux procédures distantes à la compilation du déclencheur. Or les conteneurs des sites démarrent après le global : au moment où le déclencheur est compilé, les procédures distantes n'existent pas encore, et un appel direct échouerait avec l'erreur PLS-00352. En passant par du SQL dynamique (EXECUTE IMMEDIATE), la résolution est **différée à l'exécution**, ce qui rompt cette dépendance de compilation.

**2. PRAGMA AUTONOMOUS\_TRANSACTION.** Les procédures distantes effectuent un COMMIT. Sans cette directive, ce COMMIT se produirait au sein de la transaction distribuée pilotée par le déclencheur, ce qui déclencherait l'erreur ORA-02089 (« COMMIT interdit dans une session subordonnée »). La directive isole chaque appel de site dans sa **propre transaction autonome**.

**Contrepartie assumée.** Cette autonomie transactionnelle a un prix : si la transaction globale est ensuite annulée (ROLLBACK), le site a déjà validé sa copie de la ligne. Il n'y a donc **pas d'atomicité distribuée**. Ce compromis est acceptable dans un cadre pédagogique ; un système de production exigerait une validation en deux phases (2PC/XA) ou une file de messages fiable (Oracle Advanced Queuing). Ce point est repris en section 12.

### 6.3 Le cas subtil de la mise à jour

Une mise à jour peut faire *entrer* une ligne dans un fragment, l'en faire *sortir*, ou l'y *maintenir*. Le déclencheur de mise à jour compare donc l'appartenance au fragment *avant* et *après* la modification, et choisit l'action adéquate parmi quatre cas, pour chaque site (tableau 5).

| Avant            | Après            | Action sur le site                       |
|------------------|------------------|------------------------------------------|
| dans le fragment | dans le fragment | updateligne (mise à jour sur place)      |
| dans le fragment | hors fragment    | deleteligne (la ligne quitte le site)    |
| hors fragment    | dans le fragment | insertligne (la ligne entre sur le site) |
| hors fragment    | hors fragment    | aucune action                            |

TABLE 5 – Les quatre cas gérés par SYNC\_UPDATE\_LIGNE, par site.

```

1 v_in_s1_old := (v_old_categ = 50 AND :OLD.QUANTITE > 100);
2 v_in_s1_new := (v_new_categ = 50 AND :NEW.QUANTITE > 100);
3
4 IF v_in_s1_old AND v_in_s1_new THEN
5     EXECUTE IMMEDIATE 'BEGIN pdb_admin.updateligne@link_to_site1(:1,:2,:3,:4); END;'
6     USING :NEW.IDLIGNECOMMANDE, :NEW.IDPRODUIT, :NEW.QUANTITE, :NEW.REMISE;
7 ELSIF v_in_s1_old AND NOT v_in_s1_new THEN
8     EXECUTE IMMEDIATE 'BEGIN pdb_admin.deleteligne@link_to_site1(:1); END;'
9     USING :OLD.IDLIGNECOMMANDE;
10 ELSIF NOT v_in_s1_old AND v_in_s1_new THEN
11     EXECUTE IMMEDIATE 'BEGIN pdb_admin.insertligne@link_to_site1(:1,:2,:3,:4,:5); END;'
12     USING :NEW.IDLIGNECOMMANDE, :NEW.IDCOMMANDE, :NEW.IDPRODUIT,
13         :NEW.QUANTITE, :NEW.REMISE;
14 END IF;
```

Extrait 11 – Logique de transition de fragment à la mise à jour (extrait, partie Site 1).

Dans le scénario 2, cette logique se simplifie en une comparaison du seul franchissement du seuil QUANTITE = 100, puisque le routage ne dépend plus de la catégorie.

## 7 Requêtes distribuées et optimisation (Q5 et Q6)

### 7.1 Analyse et optimisation d'une requête locale (Q5)

La requête analytique étudiée compte le nombre de commandes par client pour l'année 2026. Sur un schéma « froid » (sans index), le plan d'exécution révèle deux TABLE ACCESS FULL (balayages complets) et une jointure par hachage coûteuse. Deux leviers d'optimisation sont appliqués :

1. **Réécriture du prédicat de date.** La forme `EXTRACT(YEAR FROM DATECOMMANDE)=2026` empêche l'usage d'un index B-tree (la valeur indexée n'est pas la valeur calculée). On la remplace par un prédicat d'intervalle `DATECOMMANDE >= DATE '2026-01-01' AND < DATE '2027-01-01'`, qui autorise un INDEX RANGE SCAN.
2. **Création d'index** sur `COMMANDES(DATECOMMANDE)` et `COMMANDES(IDCLIENT)`, ce qui permet à l'optimiseur de remplacer le balayage complet par un parcours d'index et la jointure par hachage par une jointure par boucles imbriquées (NESTED LOOPS).

```

1  -- Plan AVANT optimisation
2  EXPLAIN PLAN SET STATEMENT_ID = 'before_index' FOR
3  SELECT c.IDCLIENT, c.SOCIETE, COUNT(cmd.IDCOMMANDE) AS NB_COMMANDES
4  FROM   CLIENTS c JOIN COMMANDES cmd ON c.IDCLIENT = cmd.IDCLIENT
5  WHERE  EXTRACT(YEAR FROM cmd.DATECOMMANDE) = 2026      -- bloque l'index
6  GROUP BY c.IDCLIENT, c.SOCIETE
7  ORDER BY NB_COMMANDES DESC;
8
9  SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY(
10     filter_preds => 'STATEMENT_ID = ''before_index'''));
11
12  -- Index correctifs
13  CREATE INDEX idx_commandes_date      ON COMMANDES(DATECOMMANDE);
14  CREATE INDEX idx_commandes_idclient  ON COMMANDES(IDCLIENT);
15
16  -- Requete optimisee : predicat d'intervalle => INDEX RANGE SCAN
17  EXPLAIN PLAN SET STATEMENT_ID = 'after_index' FOR
18  SELECT c.IDCLIENT, c.SOCIETE, COUNT(cmd.IDCOMMANDE) AS NB_COMMANDES
19  FROM   CLIENTS c JOIN COMMANDES cmd ON c.IDCLIENT = cmd.IDCLIENT
20  WHERE  cmd.DATECOMMANDE >= DATE '2026-01-01'
21  AND    cmd.DATECOMMANDE < DATE '2027-01-01'
22  GROUP BY c.IDCLIENT, c.SOCIETE
23  ORDER BY NB_COMMANDES DESC;

```

Extrait 12 – Capture d'un plan d'exécution et création d'index (extrait de 07\_query\_analysis.sql).

### 7.2 Reconstruction d'une vue globale (Q6)

Pour calculer le chiffre d'affaires 2026 par catégorie, on recombine les deux fragments distants au travers des liens de base de données. Le revenu d'une ligne vaut  $QUANTITE \times PRIXUNITAIRE \times (1 - REMISE)$ . On emploie UNION ALL (et non UNION) : les fragments étant disjoints par construction, aucune ligne n'apparaît deux fois, et UNION ALL évite la passe de déduplication superflue.

```

1  SELECT IDCATEG, ROUND(SUM(CA_LIGNE), 2) AS CHIFFRE_AFFAIRES_2026
2  FROM (
3      -- Contribution du Site 1
4      SELECT p.IDCATEG,
5             lc.QUANTITE * p.PRIXUNITAIRE * (1 - NVL(lc.REMISE, 0)) AS CA_LIGNE
6      FROM   pdb_admin.LIGNECOMMANDES1@link_to_site1 lc
7      JOIN   pdb_admin.PRODUITS1@link_to_site1 p ON lc.IDPRODUIT = p.IDPRODUIT
8      JOIN   pdb_admin.COMMANDES1@link_to_site1 c ON lc.IDCOMMANDE = c.IDCOMMANDE
9      WHERE  c.DATECOMMANDE >= DATE '2026-01-01'
10     AND    c.DATECOMMANDE < DATE '2027-01-01'

```

```

11
12     UNION ALL                -- fragments disjoints : pas de doublon possible
13
14     -- Contribution du Site 2
15     SELECT p.IDCATEG,
16            lc.QUANTITE * p.PRIXUNITAIRE * (1 - NVL(lc.REMISE, 0)) AS CA_LIGNE
17     FROM   pdb_admin.LIGNECOMMANDES@link_to_site2 lc
18     JOIN   pdb_admin.PRODUITS2@link_to_site2 p ON lc.IDPRODUIT = p.IDPRODUIT
19     JOIN   pdb_admin.COMMANDES2@link_to_site2 c ON lc.IDCOMMANDE = c.IDCOMMANDE
20     WHERE  c.DATECOMMANDE >= DATE '2026-01-01'
21     AND    c.DATECOMMANDE <  DATE '2027-01-01'
22 )
23 GROUP BY IDCATEG
24 ORDER BY CHIFFRE_AFFAIRES_2026 DESC;

```

Extrait 13 – Chiffre d'affaires distribué reconstruit via les liens de base de données (extrait de 08\_distributed\_query.sql).

## 8 Nœud de sauvegarde

Un quatrième nœud assure une **copie de secours** du jeu de données global. Il héberge un schéma miroir (sans clés étrangères, volontairement) et une procédure `sync_from_global` qui effectue un **rechargement complet** : vidage de toutes les tables (`TRUNCATE`, des feuilles vers la racine) puis réinsertion intégrale depuis le global (de la racine vers les feuilles, pour respecter l'ordre des dépendances). Cette procédure est planifiée toutes les heures par `DBMS_SCHEDULER`.

```

1 CREATE OR REPLACE PROCEDURE pdb_admin.sync_from_global AS
2 BEGIN
3     -- Vidage des enfants vers les parents
4     EXECUTE IMMEDIATE 'TRUNCATE TABLE pdb_admin.LIGNECOMMANDES';
5     EXECUTE IMMEDIATE 'TRUNCATE TABLE pdb_admin.COMMANDES';
6     -- ... (autres tables)
7
8     -- Rechargement des parents vers les enfants
9     INSERT INTO pdb_admin.CATEGORIES SELECT * FROM pdb_admin.CATEGORIES@link_to_global;
10    -- ...
11    INSERT INTO pdb_admin.LIGNECOMMANDES SELECT * FROM
12    pdb_admin.LIGNECOMMANDES@link_to_global;
13    COMMIT;
14 END sync_from_global;
15 /
16
17 BEGIN
18     DBMS_SCHEDULER.CREATE_JOB(
19         job_name      => 'BACKUP_SYNC_JOB',
20         job_type       => 'STORED_PROCEDURE',
21         job_action      => 'pdb_admin.sync_from_global',
22         start_date      => SYSTIMESTAMP,
23         repeat_interval => 'FREQ=HOURLY;INTERVAL=1',
24         enabled         => TRUE);
25 END;
26 /

```

Extrait 14 – Procédure de resynchronisation et job horaire (extrait de 04\_create\_sync\_job.sql).

## 9 Tableau de bord web

Une application **Next.js** (App Router, React 19) offre une interface d'observation et de test. Côté serveur, des *routes API* se connectent à Oracle via le pilote `node-oracledb`, en gérant

un **pool de connexions** par nœud. Le pool est mémorisé dans `globalThis` pour survivre aux rechargements à chaud du serveur de développement.

```

1  async function ensurePool(alias: NodeAlias) {
2    if (g._oraPools!.has(alias)) return;
3    try {
4      await oracledb.createPool({
5        poolAlias: alias,
6        ...configs[alias](),
7        poolMin: 0, poolMax: 4, poolIncrement: 1,
8        poolTimeout: 60,
9        connectTimeout: 5, // secondes ; evite les fuites de connexion
10     });
11     g._oraPools!.add(alias);
12   } catch (err: unknown) {
13     const msg = err instanceof Error ? err.message : String(err);
14     if (msg.includes('NJS-046') || msg.includes('already exists')) {
15       g._oraPools!.add(alias); // pool deja cree par un hot-reload
16     } else { throw err; }
17   }
18 }

```

Extrait 15 – Gestion d’un pool de connexions par nœud (extrait de `lib/oracle.ts`).

La route d’insertion illustre bien la *transparence* du dispositif : l’application **insère uniquement sur le global**, attend brièvement la propagation par les déclencheurs, puis interroge les deux sites pour *constater* où la ligne a été routée et le restituer à l’interface.

```

1  // 1) Insertion sur le noeud global : les triggers se declenchent seuls
2  await run('global',
3    'INSERT INTO LIGNECOMMANDES (IDLIGNECOMMANDE, IDCOMMANDE, IDPRODUIT, QUANTITE, REMISE)
4    VALUES (:1, :2, :3, :4, :5)',
5    [idlignecommande, idcommande, idproduit, quantite, remise]);
6
7  // 2) On laisse aux triggers le temps de propager
8  await sleep(700);
9
10 // 3) On constate sur quel(s) site(s) la ligne a atterri
11 const [s1Rows, s2Rows] = await Promise.allSettled([
12   query('site1', findSql('LIGNECOMMANDES1'), [idlignecommande]),
13   query('site2', findSql('LIGNECOMMANDES2'), [idlignecommande]),
14 ]);

```

Extrait 16 – Insertion sur le global puis observation du routage (extrait de `app/api/insert/route.ts`).

Les autres routes exposent l’état de santé des nœuds (`/api/health`, qui vérifie aussi l’état des déclencheurs et la dernière exécution du job de sauvegarde) et le contenu des fragments (`/api/fragments`). La page React rafraîchit l’état de santé toutes les dix secondes.

## 10 Démonstration et captures d’écran

Cette section regroupe les captures d’écran illustrant le système en fonctionnement : l’orchestration des conteneurs, l’interface du tableau de bord, puis le déroulement des tests de manipulation (insertion, suppression, mise à jour) qui valident la synchronisation par déclencheurs (section 6).

```

abde@oracle-project:~/db-project/scenario-1$ docker compose up -d
[+] up 19/19
✓ Image gvenzl/oracle-free:23-slim-faststart Pulled 130.1s
✓ Network scenario-1_maroc-db-net Created 0.0s
✓ Volume scenario-1_site-1-data Created 0.0s
✓ Volume scenario-1_global-data Created 0.0s
✓ Volume scenario-1_backup-data Created 0.0s
✓ Volume scenario-1_site-2-data Created 0.0s
✓ Container oracle-global Healthy 170.1s
✓ Container oracle-site-2 Started 142.8s
✓ Container oracle-backup Started 142.8s
✓ Container oracle-site-1 Started 142.7s
abde@oracle-project:~/db-project/scenario-1$ docker compose ps
NAME IMAGE COMMAND SERVICE CREATED STATUS PORTS
oracle-backup gvenzl/oracle-free:23-slim-faststart "container-entypoin..." db-backup 18 minutes ago Up 15 minutes (healthy) 0.0.0.0:1524->1521/tcp, [:::1524->1521/tcp]
oracle-global gvenzl/oracle-free:23-slim-faststart "container-entypoin..." db-global 18 minutes ago Up 17 minutes (healthy) 0.0.0.0:1521->1521/tcp, [:::1521->1521/tcp]
oracle-site-1 gvenzl/oracle-free:23-slim-faststart "container-entypoin..." db-site-1 18 minutes ago Up 15 minutes (healthy) 0.0.0.0:1522->1521/tcp, [:::1522->1521/tcp]
oracle-site-2 gvenzl/oracle-free:23-slim-faststart "container-entypoin..." db-site-2 18 minutes ago Up 15 minutes (healthy) 0.0.0.0:1523->1521/tcp, [:::1523->1521/tcp]
abde@oracle-project:~/db-project/scenario-1$

```

FIGURE 3 – Les conteneurs Oracle orchestrés par Docker Compose, tous démarrés.

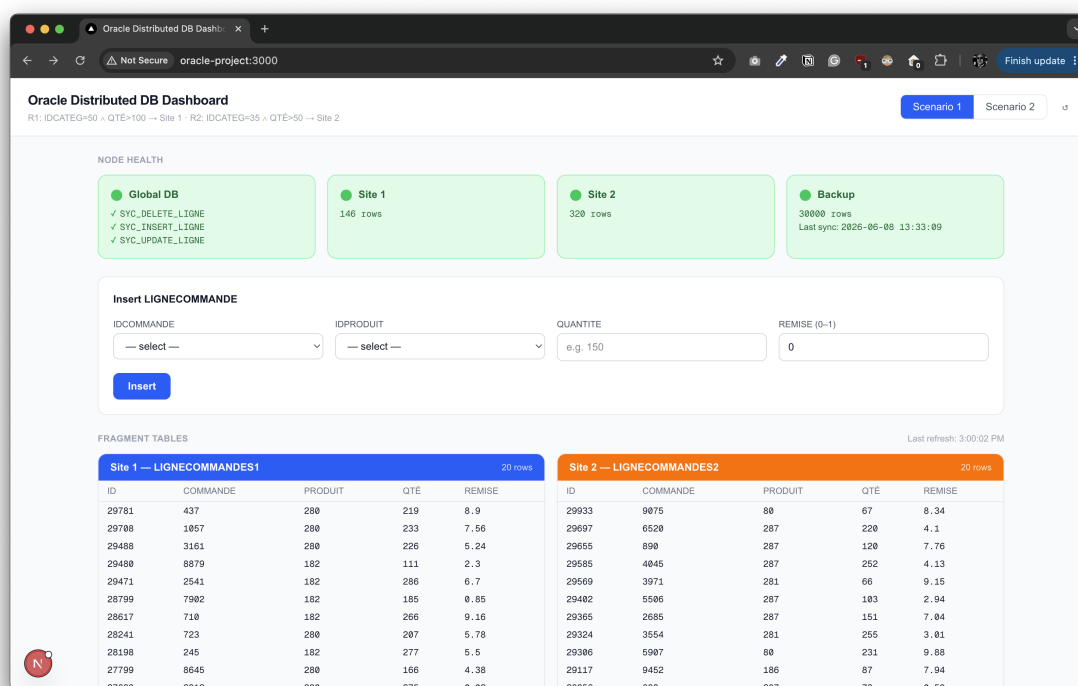


FIGURE 4 – Le tableau de bord affichant l'état des fragments répartis.

## 10.1 Conteneurs Docker en cours d'exécution

## 10.2 Tableau de bord web

## 10.3 Tests de manipulation des données

Les trois captures suivantes documentent un aller-retour complet sur la table `LIGNECOMMANDES` du nœud global et la propagation attendue vers le site propriétaire.

# 11 Avantages de l'architecture

## Transparence de localisation

L'application n'a qu'un seul point d'écriture (le global) et ignore où résident physiquement les données. Synonymes et déclencheurs masquent entièrement la répartition.

## Reproductibilité totale (infrastructure as code)

L'intégralité du système (topologie, schémas, données, logique) est décrite par des fichiers

```

abde@oracle-project: ~/db-project/scenario-1
abde@oracle-project:~/db-project/scenario-1$ docker exec -it oracle-global sqlplus pdb_admin/Admin123@//localhost:1521/G_PDB
SQL*Plus: Release 23.26.2.0.0 - Production on Mon Jun 8 14:22:25 2026
Version 23.26.2.0.0

Copyright (c) 1982, 2026, Oracle. All rights reserved.

Last Successful login time: Mon Jun 08 2026 14:16:12 +00:00

Connected to:
Oracle AI Database 26ai Free Release 23.26.2.0.0 - Develop, Learn, and Run for Free
Version 23.26.2.0.0

SQL> SET LINESIZE 200
SQL> SET PAGESIZE 50
SQL> SELECT * FROM (SELECT lc.IDCOMMANDE, lc.IDPRODUIT FROM pdb_admin.LIGNECOMMANDES lc JOIN pdb_admin.PRODUITS p ON lc.IDPRODUIT=p.IDPRODUIT WHERE p.IDCATEG=50 AND lc.QUANTITE>100) WHERE ROWNUM=1;

IDCOMMANDE  IDPRODUIT
-----
575         182

SQL> INSERT INTO pdb_admin.LIGNECOMMANDES (IDLIGNECOMMANDE, IDCOMMANDE, IDPRODUIT, QUANTITE, REMISE) VALUES (99001, 575, 182, 150, 0);

1 row created.

SQL> COMMIT;

Commit complete.

SQL> SELECT COUNT(*) FROM pdb_admin.LIGNECOMMANDES2@link_to_site2 WHERE IDLIGNECOMMANDE=99001;

COUNT(*)
-----
0

SQL>

```

```

abde@oracle-project: ~/db-project/scenario-1
abde@oracle-project:~/db-project/scenario-1$ docker exec -it oracle-site-1 sqlplus pdb_admin/Admin123@//localhost:1521/S1_PDB
SQL*Plus: Release 23.26.2.0.0 - Production on Mon Jun 8 14:16:18 2026
Version 23.26.2.0.0

Copyright (c) 1982, 2026, Oracle. All rights reserved.

Last Successful login time: Mon Jun 08 2026 13:59:29 +00:00

Connected to:
Oracle AI Database 26ai Free Release 23.26.2.0.0 - Develop, Learn, and Run for Free
Version 23.26.2.0.0

SQL> SET LINESIZE 200
SQL> SET PAGESIZE 50
SQL> SELECT * FROM pdb_admin.LIGNECOMMANDES1 WHERE IDLIGNECOMMANDE=99001;

IDLIGNECOMMANDE IDCOMMANDE  IDPRODUIT  QUANTITE  REMISE
-----
99001          575         182        150        0

SQL>

```

FIGURE 5 – Test d’insertion et propagation vers le site.

```

SQL> DELETE FROM pdb_admin.LIGNECOMMANDES WHERE IDLIGNECOMMANDE=99001;

1 row deleted.

SQL> COMMIT;

Commit complete.

SQL>

```

```

SQL> SELECT COUNT(*) AS still_here FROM pdb_admin.LIGNECOMMANDES1 WHERE IDLIGNECOMMANDE=99001;

STILL_HERE
-----
0

SQL>

```

FIGURE 6 – Test de suppression et nettoyage en cascade.

versionnés. Un `docker compose up` reconstruit l’environnement à l’identique; les scripts `purge.sh` et `rsite-1.sh` facilitent la remise à zéro.

### Scripts idempotents

L’emploi systématique de `CREATE OR REPLACE` et de blocs `DROP ... EXCEPTION WHEN OTHERS` rend les scripts ré-exécutables sans erreur.

### Intégrité référentielle préservée par fragment

Le rapatriement automatique des lignes parentes et la reconstruction des clés garantissent que chaque site reste un sous-schéma cohérent et interrogeable de façon autonome.

### Découplage à la compilation

Le recours à `EXECUTE IMMEDIATE` affranchit le système de l’ordre de démarrage des conteneurs, point notoirement délicat des déploiements distribués.

### Optimisation mesurable

La démarche `EXPLAIN PLAN` avant/après objective le gain apporté par les index et la réécriture des prédicats.

### Reconstruction efficace

La disjonction garantie des fragments autorise un `UNION ALL` sans déduplication pour reconstituer la vue globale.

### Observabilité

Le tableau de bord et le job de sauvegarde horaire offrent un suivi opérationnel immédiat de l’état du système.

```

SQL> INSERT INTO pdb_admin.LIGNECOMMANDES (IDLIGNECOMMANDE, IDCOMMANDE, IDPRODUIT,
QUANTITE, REMISE) VALUES (99005, 575, 182, 101, 0);
1 row created.
SQL> COMMIT;
Commit complete.
SQL> UPDATE pdb_admin.LIGNECOMMANDES SET QUANTITE=50 WHERE IDLIGNECOMMANDE=99005;
1 row updated.
SQL> COMMIT;
Commit complete.
SQL> SELECT QUANTITE FROM pdb_admin.LIGNECOMMANDES WHERE IDLIGNECOMMANDE=99005;
   QUANTITE
-----
         50
SQL>

SQL> SELECT * FROM pdb_admin.LIGNECOMMANDES1 WHERE IDLIGNECOMMANDE=99005;
IDLIGNECOMMANDE IDCOMMANDE IDPRODUIT QUANTITE REMISE
-----
          99005         575         182         101         0
SQL> SELECT COUNT(*) FROM pdb_admin.LIGNECOMMANDES1 WHERE IDLIGNECOMMANDE=99005;
COUNT(*)
-----
         0
SQL>

```

FIGURE 7 – Test de **mise à jour** (changement de fragment).

## 12 Inconvénients et limites

### Absence d'atomicité distribuée

C'est la limite majeure. La directive `PRAGMA AUTONOMOUS_TRANSACTION` fait que les écritures de site sont validées indépendamment de la transaction globale : un `ROLLBACK` sur le global *ne défait pas* une ligne déjà inscrite sur un site. En cas d'échec partiel, les fragments peuvent diverger du global. Une vraie cohérence exigerait une validation en deux phases (2PC/XA) ou une file de messages transactionnelle (Oracle AQ).

### Point de défaillance unique (SPOF)

Tout le routage repose sur le nœud global et ses déclencheurs. Si le global est indisponible, plus aucune écriture ne peut être propagée ; les sites ne sont pas autonomes en écriture.

### Sauvegarde grossière (RPO élevé)

Le nœud de secours procède par rechargement complet (`TRUNCATE` + réinsertion) toutes les heures : la perte de données potentielle peut atteindre une heure, le coût est proportionnel au volume total (et non aux changements), et le `TRUNCATE` (DDL auto-validé) laisse transitoirement les tables vides, sans cohérence de lecture pendant la resynchronisation.

### Sécurité perfectible

Les mots de passe figurent en clair dans les scripts et le fichier Compose. Les habilitations sont accordées par `GRANT` directs plutôt que par rôles (le script de rôles est d'ailleurs laissé en commentaire), et un compte de service unique (`g_user`) est partagé : le principe de moindre privilège n'est pas respecté.

### Génération de clés primaires fragile

Le tableau de bord calcule la prochaine clé par `MAX(IDLIGNECOMMANDE)+1`, ce qui est sujet aux conditions de course en cas d'insertions concurrentes ; une séquence Oracle serait préférable.

### Propagation observée par temporisation

La route d'insertion attend 700 ms « en aveugle » avant de constater le routage : ce délai arbitraire peut s'avérer trop court (faux négatif) ou inutilement long.

### Coût des jointures distribuées

Les requêtes Q6 rapatrient les lignes des sites au travers des liens de base de données ; sur de gros volumes, ces aller-retours réseau deviennent un goulot d'étranglement.

### Asymétrie du scénario 1

Sa fragmentation n'étant pas exhaustive, une part des lignes n'existe que sur le global, ce qui complique le raisonnement (une insertion peut n'aboutir sur aucun site).

## 13 Pistes d'amélioration

1. **Cohérence forte** : remplacer les transactions autonomes par une validation en deux phases, ou adopter une réplication par *vues matérialisées* avec rafraîchissement à la validation (REFRESH FAST ON COMMIT) ou par Oracle GoldenGate.
2. **Séquences Oracle** pour la génération des identifiants, en lieu et place de MAX+1.
3. **Sécurité** : rôles applicatifs à privilèges minimaux, secrets gérés hors du dépôt (Docker secrets, coffre-fort), comptes de service distincts par usage.
4. **Sauvegarde incrémentale** fondée sur la capture de changements (*materialized view logs*, CDC) ou sur RMAN, afin de réduire le RPO et le coût.
5. **Confirmation de propagation** fondée sur un événement ou un sondage borné plutôt que sur un délai fixe dans le tableau de bord.
6. **Autonomie des sites** : autoriser des écritures locales avec réconciliation, pour atténuer la dépendance au nœud global.

## 14 Conclusion

Ce projet met en œuvre, de bout en bout, une chaîne complète de gestion de données réparties : fragmentation horizontale paramétrable (par catégorie ou par volume), procédures stockées garantissant l'intégrité de chaque fragment, synchronisation automatique par déclencheurs avec routage transparent, optimisation de requêtes objectivée par les plans d'exécution, reconstruction distribuée du chiffre d'affaires, sauvegarde planifiée et tableau de bord d'observation. L'ensemble est entièrement reproductible grâce à Docker Compose et à des scripts idempotents.

Les choix techniques les plus instructifs (le SQL dynamique pour briser la dépendance de compilation, les transactions autonomes pour permettre la validation distante) illustrent bien les arbitrages propres aux systèmes distribués : ici, la *disponibilité* et la *simplicité* de mise en œuvre ont été privilégiées au détriment de la *cohérence forte*. Cette tension, caractéristique des bases de données réparties, constitue précisément le principal enseignement du projet, et trace la feuille de route d'une éventuelle version de production : validation en deux phases, sécurité durcie et sauvegarde incrémentale.